R1.01 - Maîtrise des Structures Répétitives, Preuve de Terminaison et Invariants de Boucle

Thème principal : Algorithmique Structurée et Assurance Qualité Logicielle (Itérations, Variants et Invariants)

Synthèse exécutive : Ce cours pose les bases fondamentales de l'itération en algorithmique structurée. Son objectif est d'éliminer définitivement l'anti-pattern du copier-coller d'instructions pour pouvoir traiter des jeux de données de taille dynamique. Le cours fournit une grille de décision claire pour choisir le bon type de boucle (POUR, TANT QUE, RÉPÉTER... JUSQU'À) selon le contexte d'exécution. Enfin, il introduit les méthodes formelles d'assurance qualité logicielle indispensables en BUT Informatique : le 'variant de boucle' pour démontrer scientifiquement la terminaison d'un programme (absence de boucle infinie) et 'l'invariant de boucle' pour prouver la correction mathématique du résultat produit.

1. Typologie des Boucles et Critères de Choix Algorithmique

En programmation, une structure de contrôle itérative permet de répéter un traitement un certain nombre de fois sans dupliquer le texte du code source. Le choix d'une structure itérative repose sur deux questions logiques fondamentales : 1. Connaît-on le nombre d'itérations à l'avance ? Si le nombre de tours est déterminé avant le début du traitement (ex: parcourir un tableau de 50 éléments ou traiter N étudiants), on utilise une boucle 'POUR'. La structure gère automatiquement l'initialisation, l'incrémentation de l'indice et le test de sortie. 2. Si le nombre de tours dépend d'un événement dynamique, combien de fois au minimum le traitement doit-il s'exécuter ? Si la condition peut être fausse dès le début et que le traitement ne doit pas s'exécuter dans ce cas, on utilise la boucle 'TANT QUE' (0 à N exécutions possible). Si le traitement doit obligatoirement être appliqué au moins une fois avant de pouvoir tester la condition de sortie (ex: afficher un menu interactivement ou effectuer un contrôle de saisie utilisateur), on utilise la boucle 'RÉPÉTER... JUSQU'À' (1 à N exécutions).

- Anti-pattern du Copier-Coller : Dupliquer du code le rend rigide, illisible et inmaintenable face à des volumes de données variables.
- Itération Bornée (POUR) : Structure employée lorsque le nombre d'itérations est fixe et connu avant d'entrer dans la boucle.
- Itération Non Bornée à Pré-condition (TANT QUE) : La condition logique est évaluée AVANT chaque tour ; le bloc s'exécute de 0 à N fois.
- Itération Non Bornée à Post-condition (RÉPÉTER... JUSQU'À) : Le bloc d'instructions s'exécute D'ABORD, puis la condition d'arrêt est testée ; garantit au moins 1 exécution (1 à N fois).

Cas concret enseignant : *L'analogie du coureur sur la piste d'athlétisme : Si l'entraîneur dit 'Fais 10 tours de piste', c'est une boucle POUR. S'il dit 'Cours tant qu'il ne pleut pas', c'est un TANT QUE (s'il pleut déjà lorsqu'il est au vestiaire, le coureur ne commence même pas = 0 tour). S'il dit 'Fais un tour et arrête-toi dès que tu es essoufflé', c'est un RÉPÉTER... JUSQU'À (le coureur est contraint d'effectuer au moins un premier tour avant de vérifier son niveau d'essoufflement).*

2. Preuve Formelle de Terminaison : Le Variant de Boucle

Pour prouver scientifiquement qu'un algorithme ne restera pas bloqué indéfiniment dans une boucle non bornée, l'informaticien doit formaliser un 'variant de boucle'. Mathématiquement, la preuve s'appuie sur le théorème du bon ordre des entiers naturels : il est impossible de construire une suite d'entiers naturels qui soit à la fois infinie, strictement décroissante et minorée. Pour établir une preuve de terminaison valide, il faut démontrer deux propriétés : 1. Décroissance stricte : À chaque itération, la valeur du variant diminue strictement d'au moins une unité ($V_{k+1} \le V_k - 1$). 2. Minoration / Seuil : Il existe une constante entière M (souvent 0) telle que dès que $V \le M$, la condition de continuation de la boucle devient obligatoirement fausse. Si ces deux critères sont remplis, la terminaison de la boucle en un nombre fini d'étapes est mathématiquement garantie.

• Boucle Infinie : Dysfonctionnement critique provoquant le gel complet d'une application lorsque la condition de sortie n'est jamais atteinte.

• Variant de Boucle : Grandeur entière calculée sur les variables de l'algorithme qui mesure la 'distance' restante jusqu'à l'arrêt.

• Décroissance Stricte : Propriété imposant au variant de diminuer d'au moins 1 entier à chaque passage dans la boucle (V_{k+1} < V_k).

• Minorant : Valeur plancher (généralement 0) en dessous de laquelle le variant ne peut descendre sans violer la condition de maintien.

Cas concret enseignant : *La métaphore de la jauge d'essence d'une voiture : À chaque kilomètre parcouru (une itération), le volume de carburant dans le réservoir (le variant) diminue obligatoirement d'une quantité stricte. Comme le réservoir ne peut pas descendre en dessous de 0 litre (le minorant), la voiture finira obligatoirement par s'arrêter. Si la jauge ne baissait pas à chaque kilomètre parcouru, la voiture roulerait à l'infini : c'est l'équivalent direct du bug de la boucle infinie.*

3. Preuve Formelle de Correction : L'Invariant de Boucle

Prouver qu'une boucle s'arrête (terminaison) est nécessaire mais insuffisant : il faut également prouver qu'elle produit le résultat exact attendu (correction). C'est le rôle de l'invariant de boucle, dont la logique s'apparente directement à une démonstration par récurrence : 1. Initialisation : On établit que la propriété P est vraie avant d'entrer dans la boucle ($k=0$). 2. Conservation : On suppose P vraie au début d'une itération k. On montre qu'après exécution des instructions du corps de la boucle, P demeure vraie pour l'étape $k+1$. 3. Conclusion : Lorsque la boucle se termine, la condition de maintien C devient fausse (soit $\neg C$). Puisque l'invariant P est toujours vrai, la conjonction logique $(P \land \neg C)$ permet de déduire formellement que la variable de sortie contient la valeur exacte exigée par le problème.

• Invariant de Boucle : Assertion ou propriété logique qui demeure VRAIE avant le début de la boucle, après chaque itération, et à la sortie.

• Initialisation : Démonstration que l'invariant est vrai dès l'état initial des variables, avant la première itération.

• Conservation (Hérédité) : Démonstration que si l'invariant est vrai au début du tour k, il le reste à la fin du tour k après la mise à jour des variables.

• Conclusion (Correction) : Combinaison de l'invariant conservé et de la négation de la condition de boucle, prouvant que le résultat final calculé correspond exactement à la spécification.

Cas concret enseignant : *L'analogie de la tournée du livreur de journaux : Au départ de sa tournée matinale, l'égalité 'Journaux distribués + Journaux restants dans le sac = Total initial' est vraie. À chaque maison servie, les deux quantités changent mais leur somme reste rigoureusement la même tout au long de la journée (invariant). Quand le sac est vide à la fin de la tournée (condition de sortie), le nombre de journaux distribués est obligatoirement égal au total initial.*

■ Questions & Interventions des Étudiants en Classe

Question étudiant : Est-ce qu'on peut toujours remplacer une boucle POUR par une boucle TANT QUE ?
Réponse du professeur : Oui, absolument. Toute boucle 'POUR' peut être réécrite sous forme de boucle 'TANT QUE' en gérant explicitement l'initialisation de la variable de contrôle avant la boucle, la condition de maintien (ex: i <= N) et l'incrémentation du compteur à la fin du corps de la boucle. En revanche, l'inverse n'est pas vrai : on ne peut pas remplacer une boucle TANT QUE par une boucle POUR lorsque le nombre total d'itérations n'est pas connu au préalable.

Question étudiant : Si j'utilise une boucle TANT QUE avec la condition (i < N), est-ce que les instructions s'exécutent si i est déjà égal à N dès le début ?
Réponse du professeur : Non, elles ne s'exécuteront pas du tout (0 itération). La boucle TANT QUE est à 'pré-condition', ce qui signifie que l'expression logique est évaluée AVANT la moindre exécution du corps. Si 'i' vaut initialement N, la condition (i < N) est évaluée à FAUX d'entrée, et le programme saute immédiatement après le bloc de boucle sans l'exécuter.

■ Exercice & Démonstration d'Application

Énoncé : Écrire un algorithme en pseudo-code permettant de calculer la somme des N premiers entiers pairs stricts (par exemple pour N=3, sommer 2 + 4 + 6 = 12). Justifier le choix de la structure itérative, rédiger le code propre et formaliser le variant de boucle garantissant la terminaison.

Résolution pas à pas :
Étape 1 : Choix de la structure répétitive. Le paramètre N est fourni en entrée du programme ; le nombre total d'itérations est donc parfaitement déterminé et connu d'avance. La boucle 'POUR' est la structure la plus adaptée et la plus sécurisée. Étape 2 : Définition des variables et initialisation. - N : Entier (nombre de termes à sommer) - i : Entier (compteur de boucle) - somme : Entier (accumulateur, initialisé à 0) - terme : Entier (valeur du terme pair courant, initialisé à 2) Étape 3 : Pseudo-code algorithmique. Algorithme SommePremiersPairs Variables : N, i, somme, terme : Entiers Début Lire(N) somme <- 0 terme <- 2 Pour i allant de 1 à N par pas de 1 Faire somme <- somme + terme terme <- terme + 2 FinPour Afficher(somme) Fin Étape 4 : Démonstration du variant de boucle. Si l'on traduit cette itération sous forme de boucle TANT QUE équivalente contrôlée par le compteur i (avec i initialisé à 1 et condition i <= N), l'expression entière V = (N - i + 1) constitue un variant de boucle valide. - À chaque itération k, i est incrémenté de 1, donc V_{k+1} = N - (i + 1) + 1 = V_k - 1 (décroissance stricte d'au moins 1). - La boucle continue tant que i <= N, c'est-à-dire tant que V >= 1. Dès que V atteint 0 (minorant), la condition devient fausse et la boucle s'arrête.

Résultat : Pour N = 3 : - Tour 1 (i=1) : somme = 0 + 2 = 2 ; terme devient 4. - Tour 2 (i=2) : somme = 2 + 4 = 6 ; terme devient 6. - Tour 3 (i=3) : somme = 6 + 6 = 12 ; terme devient 8. Sortie de boucle. Valeur affichée = 12. L'algorithme se termine de façon prouvée en exactement N itérations et produit le résultat mathématique correct.

■ Auto-Évaluation Flash d'Examen

Q : Quelle est la différence fondamentale d'exécution en mémoire entre une boucle TANT QUE et une boucle RÉPÉTER... JUSQU'À ?

R : La boucle TANT QUE teste la condition d'arrêt AVANT d'exécuter le corps (pré-condition, s'exécute de 0 à N fois). La boucle RÉPÉTER... JUSQU'À s'exécute d'abord une première fois avant d'évaluer la condition d'arrêt en fin de tour (post-condition, s'exécute au moins 1 fois, de 1 à N).

Q : Quelles sont les deux propriétés mathématiques obligatoires que doit vérifier une grandeur entière pour constituer un variant de boucle valide ?

R : Elle doit être strictement décroissante à chaque itération (V_{k+1} < V_k) et elle doit être minorée par une constante entière (généralement 0), garantissant l'arrêt de la boucle dès que le seuil est atteint.

Q : Quel oubli classique dans la rédaction d'une boucle TANT QUE entraîne une boucle infinie et fait perdre la totalité des points en partiel de R1.01 ?

R : L'oubli de modifier l'état des variables intervenant dans la condition de maintien (par exemple oublier d'incrémenter le compteur i <- i + 1 à l'intérieur du corps de la boucle), ce qui rend la condition toujours vraie.

Fiche académique générée et vérifiée par Kai pour Matéi • Hébergée sur http://88.96.47.180:8765